

Lista de Exercícios

Aula 09 . Classes abstratas e Interface

Disciplina	Programação Orientada a Objetos		
Professor	Chauã Queirolo		
Entrega	-	Valor	-

Resumo

Em Programação Orientada a Objetos, classes abstratas e interfaces são recursos usados para organizar melhor o código, definir comportamentos obrigatórios e permitir o uso de polimorfismo. Elas são muito importantes quando queremos criar uma estrutura comum para várias classes, mas sem obrigar todas a terem exatamente a mesma implementação.

Uma **classe abstrata** é uma classe que serve como modelo para outras classes. Ela não deve ser usada diretamente para criar objetos. Em Dart, uma classe abstrata é criada com a palavra-chave `abstract`. Normalmente, ela representa uma ideia geral, como `Animal`, `Pessoa`, `Funcionario`, `Veiculo` ou `Personagem`.

```
abstract class Animal {  
    String nome;  
  
    Animal(this.nome);  
  
    void emitirSom(); // Método abstrato  
}
```

Nesse exemplo, `Animal` é uma classe abstrata. Ela possui o atributo `nome` e declara o método `emitirSom()`, mas não define como o som será emitido. Isso fica sob responsabilidade das classes filhas.

```
class Cachorro extends Animal {  
    Cachorro(String nome) : super(nome);  
  
    @override  
    void emitirSom() {  
        print('Au au');  
    }  
}
```

A classe `Cachorro` herda de `Animal` usando `extends`. Como `Animal` possui um método abstrato, `Cachorro` é obrigada a implementar esse método. O uso de `@override` indica que o método está sendo sobrescrito.

A principal vantagem da classe abstrata é permitir que a classe pai defina uma estrutura comum. Ela pode ter atributos, construtores, métodos concretos e métodos abstratos. Um método concreto já possui implementação. Um método abstrato apenas define a assinatura e obriga as classes filhas a implementarem seu próprio comportamento.

Uma **interface** define um contrato. Ela determina quais métodos uma classe deve possuir, mas não representa necessariamente uma relação de herança do tipo “é um”. Em Dart, qualquer classe pode ser usada como interface, mas é comum criar uma classe abstrata contendo apenas as assinaturas dos métodos.

```
abstract interface class Ligavel {  
    void ligar();  
    void desligar();  
}
```

Nesse caso, Ligavel define que qualquer classe que implemente essa interface deverá possuir os métodos ligar() e desligar().

```
class Smartphone implements Ligavel {  
    @override  
    void ligar() {  
        print('Smartphone ligando...');  
    }  
  
    @override  
    void desligar() {  
        print('Smartphone desligando...');  
    }  
}
```

A classe Smartphone usa implements, ou seja, ela promete cumprir o contrato definido por Ligavel. Por isso, é obrigada a implementar todos os métodos da interface.

A diferença principal é que a classe abstrata é usada quando existe uma relação de herança, ou seja, quando uma classe filha “é um” tipo da classe pai. Por exemplo, Cachorro é um Animal, Gerente é um Funcionario, Carro é um Veiculo.

A interface é usada quando queremos garantir que uma classe tenha determinado comportamento, sem obrigar que ela pertença a uma mesma hierarquia. Por exemplo, Smartphone, Notebook e Televisao podem implementar Ligavel, mesmo sendo tipos diferentes de objetos.

Atividades

Questão 1

Crie uma classe abstrata `Funcionario` com os atributos `nome` e `salario`. A classe deve possuir construtor para receber os dois valores e declarar o método abstrato `exibirDados()`. Depois, crie a classe `FuncionarioComum`, que herda de `Funcionario` e sobrescreve `exibirDados()` para mostrar nome e salário. Crie também a classe `Gerente`, que herda de `Funcionario`, adiciona o atributo `setor` e sobrescreve `exibirDados()` para mostrar nome, salário e setor.

No `main()`, crie um funcionário comum e um gerente. Chame o método `exibirDados()` para os dois objetos.

Questão 2

Crie uma classe `Produto` com os atributos `nome` e `preco`. Depois, crie uma interface chamada `Exibivel`, com o método `exibir()`. Crie a classe `ProdutoPerecivel`, que herda de `Produto`, adiciona o atributo `dataValidade` e implementa a interface `Exibivel`. No método `exibir()`, mostre o nome, o preço e a data de validade do produto.

No `main()`, crie um produto perecível e chame o método `exibir()`.

Questão 3

Crie uma classe abstrata `Veiculo` com os atributos `marca`, `modelo` e `ano`. A classe deve possuir construtor e declarar o método abstrato `exibirFicha()`. Depois, crie a classe `Carro`, que herda de `Veiculo` e adiciona o atributo `quantidadePortas`. Sobrescreva `exibirFicha()` para mostrar todos os dados do carro. Crie também uma nova classe filha chamada `Moto`, que herda de `Veiculo` e adiciona o atributo `cilindradas`. Sobrescreva `exibirFicha()` para mostrar todos os dados da moto.

No `main()`, crie um carro e uma moto. Chame `exibirFicha()` para os dois objetos.

Questão 4

Crie uma classe `ContaBancaria` com os atributos `titular` e `saldo`, além dos métodos `depositar(double valor)`, `sacar(double valor)` e `exibirSaldo()`. Depois, crie uma interface chamada `Rentavel`, com o método `aplicarRendimento()`. Crie a classe `ContaPoupanca`, que herda de `ContaBancaria` e implementa `Rentavel`. A conta poupança deve possuir taxa de rendimento de 1%.

Crie também a classe `ContaCorrente`, que herda de `ContaBancaria` e implementa `Rentavel`. A conta corrente deve possuir taxa de rendimento de 5%. No método `aplicarRendimento()`, aumente o saldo conforme a taxa da conta.

No `main()`, crie uma conta poupança e uma conta corrente. Deposite valores, aplique rendimento e exiba o saldo final.

Questão 5

Crie uma classe abstrata `Pessoa` com os atributos `nome` e `idade`. A classe deve possuir construtor e declarar o método abstrato `exibirDados()`. Depois, crie a classe `Aluno`, que herda de `Pessoa` e adiciona os atributos `matricula` e `curso`. Sobrescreva `exibirDados()` para mostrar todos os dados do aluno. Crie também uma nova classe filha chamada `Professor`, que herda de `Pessoa` e adiciona os atributos `disciplina` e `salario`. Sobrescreva `exibirDados()` para mostrar todos os dados do professor.

No `main()`, crie um aluno e um professor. Chame `exibirDados()` para os dois objetos.

Questão 6

Crie uma classe Ingresso com os atributos evento e valor. Depois, crie uma interface chamada Calculavel, com o método `double valorFinal()`. Crie a classe IngressoVip, que herda de Ingresso, adiciona o atributo `valorAdicional` e implementa `Calculavel`. O método `valorFinal()` deve retornar o valor do ingresso somado ao valor adicional. Crie também a classe IngressoPromocional, que herda de Ingresso e implementa `Calculavel`. Essa classe deve possuir o atributo `desconto`. O método `valorFinal()` deve retornar o valor do ingresso com o desconto aplicado.

No `main()`, crie um ingresso VIP e um ingresso promocional. Mostre o valor final de cada um.

Questão 7

Crie uma classe abstrata Animal com o atributo `nome` e o método abstrato `emitirSom()`. Depois, crie as classes Cachorro, Gato e Passaro, todas herdando de Animal. Cada classe deve sobrescrever `emitirSom()` com uma mensagem específica:

- Cachorro: "au au"
- Gato: "miau"
- Passaro: "piu piu"

Crie também uma nova classe filha chamada Vaca, que herda de Animal e sobrescreve `emitirSom()` com a mensagem "muuu". No `main()`, crie um objeto de cada animal e mande todos emitirem som.

Questão 8

Crie uma classe Livro com os atributos `titulo` e `autor`. Depois, crie uma interface chamada Informativo, com o método `exibirInformacoes()`. Crie a classe LivroDigital, que herda de Livro, adiciona o atributo `tamanhoArquivo` e implementa `Informativo`.

No método `exibirInformacoes()`, mostre o título, o autor e o tamanho do arquivo. Crie também a classe LivroFisico, que herda de Livro e implementa `Informativo`. Essa classe deve possuir o atributo `quantidadePaginas`.

No `main()`, crie um livro digital e um livro físico. Chame `exibirInformacoes()` para os dois objetos.

Questão 9

Crie uma classe abstrata Pedido com os atributos `codigo` e `valorTotal`. A classe deve possuir construtor e declarar o método abstrato `exibirPedido()`. Depois, crie a classe PedidoDelivery, que herda de Pedido e adiciona os atributos `enderecoEntrega` e `taxaEntrega`.

Sobrescreva `exibirPedido()` para mostrar código, valor total, endereço de entrega, taxa de entrega e valor final. Crie também uma nova classe filha chamada PedidoRetirada, que herda de Pedido e adiciona o atributo `nomeCliente`. Sobrescreva `exibirPedido()` para mostrar código, valor total e nome do cliente que fará a retirada.

No `main()`, crie um pedido delivery e um pedido para retirada. Exiba os dados dos dois pedidos.

Questão 10

Crie uma classe Dispositivo com os atributos `marca` e `modelo`. Depois, crie uma interface chamada Ligavel, com os métodos `ligar()` e `desligar()`. Crie a classe Smartphone, que herda de Dispositivo, adiciona o atributo `sistemaOperacional` e implementa `Ligavel`. No método `ligar()`, mostre uma mensagem específica informando que o smartphone está ligando. No método `desligar()`, mostre uma mensagem informando que o smartphone está desligando.

Crie também a classe Notebook, que herda de Dispositivo e implementa Ligavel. Ela deve possuir o atributo memoriaRam.

No main(), crie um smartphone e um notebook. Chame ligar() e desligar() para os dois objetos.

Questão 11

Neste exercício, você deve criar uma pequena hierarquia de classes usando herança, classes abstratas e interfaces em Dart. O objetivo é praticar extends, implements, super, reaproveitamento de atributos e sobrescrita de métodos com @override.

A atividade deve explorar a diferença entre classe abstrata e interface. As classes abstratas serão usadas para representar conceitos gerais do sistema, enquanto as interfaces serão usadas para definir comportamentos obrigatórios que algumas classes deverão implementar.

Interface Atacavel

Crie uma interface chamada Atacavel. Essa interface deve definir que uma classe atacável precisa possuir um comportamento de ataque contra outro personagem. Toda classe que implementar essa interface deverá obrigatoriamente saber atacar um personagem.

Interface Magico

Crie uma interface chamada Magico. Essa interface deve definir que uma classe mágica precisa possuir um comportamento de lançar magia contra outro personagem. Toda classe que implementar essa interface deverá obrigatoriamente saber lançar magia.

Interface Flamejante

Crie uma interface chamada Flamejante. Essa interface deve definir que uma classe flamejante precisa possuir um comportamento de soltar fogo contra outro personagem. Toda classe que implementar essa interface deverá obrigatoriamente saber soltar fogo.

Classe abstrata Personagem

Crie uma classe abstrata Personagem para representar qualquer personagem do sistema. Essa classe deve possuir nome, vida e nível. O nome não pode ser vazio. A vida não pode ser negativa. O nível deve ser maior ou igual a 1. A classe deve permitir que qualquer personagem receba dano e verifique se ainda está vivo. Ao receber dano, a vida deve ser reduzida, mas nunca pode ficar abaixo de zero. Um personagem está vivo quando sua vida é maior que zero. A classe Personagem também deve obrigar suas classes filhas a implementarem uma forma própria de exibir o status do personagem.

Classe abstrata Combatente

Crie uma classe abstrata Combatente, que representa personagens capazes de combater. Essa classe deve herdar de Personagem e implementar a interface Atacavel. Além dos dados herdados de Personagem, a classe Combatente deve possuir a força do combatente. Todo combatente deve ser obrigado a atacar outro personagem, mas a forma exata do ataque deverá ser definida pelas classes filhas.

Classe Guerreiro

Crie uma classe Guerreiro, que herda de Combatente. O guerreiro deve possuir armadura. O guerreiro deve implementar o ataque comum, causando dano baseado em sua força. Além disso, o guerreiro deve possuir um ataque pesado, que causa o dobro da força como dano. Ao exibir seu status, além das informações comuns de personagem, também deve mostrar sua armadura.

Classe Arqueiro

Crie uma classe Arqueiro, que herda de Combatente. O arqueiro deve possuir uma quantidade de flechas. Ao atacar, o arqueiro só pode realizar o ataque se tiver pelo menos uma flecha disponível. Cada ataque consome uma flecha. Caso não tenha flechas, deve ser exibida uma mensagem informando que o arqueiro não pode atacar.

Ao exibir seu status, além das informações comuns de personagem, também deve mostrar a quantidade de flechas.

Classe Mago

Crie uma classe Mago, que herda de Combatente e implementa a interface Magico. O mago deve possuir mana. O mago deve implementar o ataque comum, causando dano baseado em sua força. Além do ataque comum, o mago deve conseguir lançar magia. A magia só pode ser lançada se o mago tiver pelo menos 10 pontos de mana. Ao lançar magia, o mago causa dano igual à sua força somada a 15 e consome 10 pontos de mana. Caso não tenha mana suficiente, deve ser exibida uma mensagem informando que o mago não pode lançar magia.

Ao exibir seu status, além das informações comuns de personagem, também deve mostrar sua mana.

Classe abstrata Inimigo

Crie uma classe abstrata Inimigo, que também herda de Personagem. Essa classe representa personagens inimigos e deve possuir uma recompensa. Todo inimigo deve ser obrigado a exibir o valor da recompensa concedida quando for derrotado.

Classe Goblin

Crie uma classe Goblin, que herda de Inimigo. O goblin deve possuir velocidade. Ao exibir seu status, além das informações comuns de personagem, também deve mostrar sua velocidade. O goblin também deve implementar a exibição da recompensa.

Classe Dragao

Crie uma classe Dragao, que herda de Inimigo e implementa a interface Flamejante. O dragão deve possuir poder de fogo. Ele deve conseguir soltar fogo em outro personagem, causando dano igual ao seu poder de fogo. Ao exibir seu status, além das informações comuns de personagem, também deve mostrar seu poder de fogo. O dragão também deve implementar a exibição da recompensa.

Programa principal

No programa principal, crie pelo menos um objeto de cada uma das seguintes classes: Guerreiro, Arqueiro, Mago, Goblin e Dragao.

Use os seguintes personagens:

- Thoran, o guerreiro.
- Lia, a arqueira.
- Merlin, o mago.
- Gob, o goblin.
- Ignis, o dragão.

Depois, execute a seguinte sequência de ações:

- O guerreiro deve atacar o goblin.
- O arqueiro deve atacar o goblin.
- O mago deve lançar magia no dragão.
- O dragão deve soltar fogo no guerreiro.

Ao final, exiba o status de todos os personagens. Se algum inimigo estiver derrotado, exiba também sua recompensa.

Respostas

Questão 1

```
abstract class Funcionario {  
    String nome;  
    double salario;  
  
    Funcionario(this.nome, this.salario);  
  
    void exibirDados();  
}
```

```
class FuncionarioComum extends Funcionario {  
    FuncionarioComum(String nome, double salario) : super(nome, salario);  
  
    @override  
    void exibirDados() {  
        print('Funcionário comum');  
        print('Nome: $nome');  
        print('Salário: R$ $salario');  
        print('');  
    }  
}
```

```
class Gerente extends Funcionario {  
    String setor;  
  
    Gerente(String nome, double salario, this.setor) : super(nome, salario);  
  
    @override  
    void exibirDados() {  
        print('Gerente');  
        print('Nome: $nome');  
        print('Salário: R$ $salario');  
        print('Setor: $setor');  
        print('');  
    }  
}
```

```
void main() {  
    Funcionario funcionario = FuncionarioComum('João', 2500.00);  
    Funcionario gerente = Gerente('Maria', 6000.00, 'Financeiro');  
  
    funcionario.exibirDados();  
    gerente.exibirDados();  
}
```

Questão 2

```
class Produto {  
    String nome;  
    double preco;  
  
    Produto(this.nome, this.preco);  
}
```

```
abstract class Exibivel {
    void exibir();
}

class ProdutoPerecivel extends Produto implements Exibivel {
    String dataValidade;

    ProdutoPerecivel(
        String nome,
        double preco,
        this.dataValidade,
    ) : super(nome, preco);

    @override
    void exibir() {
        print('Produto perecível');
        print('Nome: $nome');
        print('Preço: R\$ $preco');
        print('Data de validade: $dataValidade');
    }
}

void main() {
    ProdutoPerecivel produto = ProdutoPerecivel('Iogurte', 5.99, '10/06/2026');

    produto.exibir();
}
```